

Collections in Java Programming:

- The Collection in Java is a framework that provides a set of interfaces and classes to store and manipulate data efficiently.
- A Collection is defined as a single unit of objects and a framework is a ready-made architecture that provides a set of interfaces and classes.
- With the use of Collections, you can write your code quickly and efficiently. You can easily store and manipulate the data by performing operations such as searching, sorting, insertion, deletion, and traversing.
- The package `java.util.Collection` or `java.util` contains all the interfaces and their implementation known as classes and algorithms for the Collection framework. Thus, you need to import the `java.util.Collection` package to use Collection classes and interfaces in Java.

The advantages of using collections are listed below:

- By using the collections framework, you can store both homogenous and heterogeneous data.
- It also provides flexibility since its size can be expanded according to the user's requirements.
- You can perform many operations such as insertion, deletion, searching, sorting, etc. very easily by the use of methods.

▼ Collection Interface in Java

- It is at the top level of the Collection framework hierarchy.
- It is implemented by all of the collection framework's classes.
- It consists of a declaration of the methods available to all the classes that implement the collection interface.
- There are many methods of Collection interface. Listed below are some of the methods of collection interfaces method:
 - ❖ `boolean add(Object obj)`
 - ❖ `boolean addAll(Collection c)`
 - ❖ `boolean remove(Object obj)`
 - ❖ `boolean removeAll(Collection c)`
 - ❖ `int size()`
 - ❖ `void clear()`

➤ ArrayList

It is a dynamic array which means that its size can be decreased or increased as per requirement. Thus, it provides flexibility. You can randomly access the elements of the array list by using its index number.

Syntax

```
ArrayList <dataType> arraylistName=new ArrayList<>();
```

Example:

```
import java.util.*;
public class AraryListExample {
    public static void main(String []array){
        ArrayList<String> list=new ArrayList<>();

        // adding elements in an arraylist
        list.add("CSE");
        list.add("AIML");
        list.add("AIDS");
        list.add("IT");
        list.add("ECE");

        // printing elements of arraylist
        System.out.println(list);
        // accessing element of arraylist by its index number
        System.out.println(list.get(0));

        // removing element
        list.remove("AIML");
        System.out.println(list);
        System.out.println("Length of arraylist =" +list.size());

        // removing element by the index number of element.
        list.remove(0);
        System.out.println(list);

        // returns false because the element doesn't exist in arraylist.
        System.out.println(list.remove("EEE"));

        // removing all the elements of arraylist.
        list.clear();
    }
}
```

Output:

```
[CSE, AIML, AIDS, IT, ECE]
Length of arraylist=5
[CSE, AIDS, IT, ECE]
[AIDS, IT, ECE]
False
```

➤ Vector

It is similar to ArrayList, which stores elements in a dynamic array. The two key differences between ArrayList and Vector are that Vector is synchronized and has many methods that aren't included in the Collections framework.

Syntax

```
Vector <dataType> vectorName=new Vector<>();
```

Example:

```
import java.util.*;
public class VectorExample {
    public static void main(String []array){
        Vector<Float> vector=new Vector<>();

        // adding elements using add method in an arraylist
        vector.add(5.4f);
        vector.add(-4.5f);
        vector.add(9.55f);
        // adding elements using addElement method in an arraylist
        vector.addElement(6.5f);
        vector.addElement(15.76f);

        // printing vector
        System.out.println(vector);

        vector.remove(5.4f);
        System.out.println(vector);
    }
}
```

Output:

```
[5.4, -4.5, 9.55, 6.5, 15.76]
[-4.5, 9.55, 6.5, 15.76]
```

➤ Hashtable

Java Hashtable class implements a hashtable, which maps keys to values. It inherits Dictionary class and implements the Map interface.

- A Hashtable is an array of a list. Each list is known as a bucket. The position of the bucket is identified by calling the hashCode() method. A Hashtable contains values based on the key.
- Java Hashtable class contains unique elements.
- Java Hashtable class doesn't allow null key or value.
- Java Hashtable class is synchronized.

Syntax:

```
Hashtable<K,V> hashtablename=new Hashtable<K,V>();
```

K: It is the type of keys maintained by this map.

V: It is the type of mapped values.

Example:

```
import java.util.*;
public class HashDemo {
    public static void main(String[] args) {
        Hashtable<String, Integer> hashtable = new Hashtable<String, Integer>();
        // Adding elements to the hashtable
        hashtable.put("A", 1);
        hashtable.put("B", 2);
        hashtable.put("C", 3);

        // Getting values from the hashtable
        int valueA = hashtable.get("A");
        System.out.println("Value of A: " + valueA);

        // Removing elements from the hashtable
        hashtable.remove("B");

        // Enumerating the elements of the hashtable
        Enumeration<String> keys = hashtable.keys();
        while (keys.hasMoreElements()) {
            String key = keys.nextElement();
            System.out.println("Key: " + key + ", Value: " + hashtable.get(key));
        }
    }
}
```

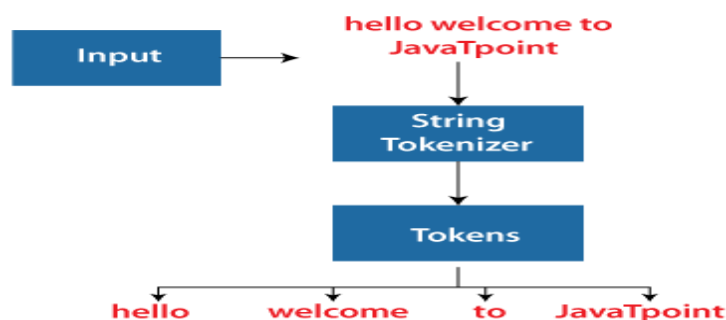
Output:

```
Value of A: 1
Key: A, Value: 1
Key: C, Value: 3
```

➤ StringTokenizer

The `java.util.StringTokenizer` class allows you to break a String into tokens. It is a simple way to break a String.

It is a legacy class of Java. In the `StringTokenizer` class, the delimiters can be provided at the time of creation or one by one to the tokens.

Example of StringTokenizer class in Java

There are 3 constructors defined in the StringTokenizer class:

StringTokenizer(String str)	It creates StringTokenizer with specified string.
StringTokenizer(String str, String delim)	It creates StringTokenizer with specified string and delimiter.
StringTokenizer(String str, String delim, boolean returnValue)	It creates StringTokenizer with specified string, delimiter and returnValue. If return value is true, delimiter characters are considered to be tokens. If it is false, delimiter characters serve to separate tokens.

The six useful methods of the StringTokenizer class are as follows:

boolean hasMoreTokens()	It checks if there is more tokens available.
String nextToken()	It returns the next token from the StringTokenizer object.
String nextToken(String delim)	It returns the next token based on the delimiter.
boolean hasMoreElements()	It is the same as hasMoreTokens() method.
Object nextElement()	It is the same as nextToken() but its return type is Object.
int countTokens()	It returns the total number of tokens.

Example:

```
import java.util.Scanner;
import java.util.StringTokenizer;

public class StringTokenizerDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string to tokenize: ");
        String inputString = scanner.nextLine();
        StringTokenizer tokenizer = new StringTokenizer(inputString, " ,");
        // Tokenize using space and comma as delimiters
        int tokenCount = tokenizer.countTokens();
        if (tokenCount > 0) {
            System.out.println("Tokens found in the input string:");
            while (tokenizer.hasMoreTokens()) {
                String token = tokenizer.nextToken();
                System.out.println("Token: " + token);
            }
            System.out.println("Total number of tokens: " + tokenCount);
        } else {
            System.out.println("No tokens found in the input string.");
        }
        scanner.close();
    }
}
```

Output:

Enter a string to tokenize: I am studying 2nd year AIML in GMRIT

Tokens found in the input string:

Token: I

Token: am

Token: studying

Token: 2nd

Token: year

Token: AIML

Token: in

Token: GMRIT

Total number of tokens: 8